

EJERCICIO TÉCNICO

Developer Lead Sr. — React Native

OCC · Redarbor México

Plazo	5 días hábiles desde la recepción de este documento
Entrega	Monorepo privado en GitHub
Stack app	Expo 54 · React Native 0.81 · TypeScript strict
Stack API	Node.js · Express

1. Contexto

Este ejercicio simula el stack tecnológico y los desafíos reales del equipo de mobile en OCC. Consiste en un monorepo con dos partes: un backend API en Node.js que sirve datos de vacantes, y una app móvil en Expo que los consume.

No hay una solución única correcta. Nos interesa entender cómo estructurar una solución, cómo tomar decisiones técnicas y cómo justificarlas. El README y la documentación arquitectónica son tan importantes como el código.

Si necesitas más tiempo por alguna razón justificada, escríbenos antes de que venza el plazo.

1.1 Estructura del monorepo

El repositorio debe tener la siguiente estructura raíz:

```
occ-lead-ejercicio/
├── app/           ← Proyecto Expo (React Native)
├── backend/       ← Servidor Node.js + Express
├── README.md      ← Instrucciones de setup y decisiones clave
└── .env.example   ← Variables de entorno necesarias
```

Cuando termines, agréranos como colaboradores del repositorio. Te compartiremos el usuario de GitHub del equipo por separado.

2. Backend API

Un servidor Express que expone una API REST con datos en memoria. No requiere base de datos: los datos se inicializan al arrancar el servidor desde un archivo de seed.

2.1 Setup

```
cd backend
npm install
npm start

# Servidor en http://localhost:3000
```

La app debe leer la URL base desde la variable de entorno `API_BASE_URL` en `app/.env`. Nunca hardcodeada.

2.2 Endpoints

Auth

Método	Endpoint	Auth	Descripción
POST	/auth/login	No	Valida credenciales mock. Devuelve JWT con expiración de 1 hora.
POST	/auth/logout	Sí	Invalida el token en servidor (blacklist en memoria).
GET	/auth/me	Sí	Valida el JWT y devuelve los datos del usuario autenticado.

Credenciales fijas: `test@occ.com.mx / Test1234`

Jobs

Método	Endpoint	Auth	Descripción
GET	/jobs	No	Lista paginada. Query params: q, city, salary_min, salary_max, sort, page, limit (default 20).
GET	/jobs/:id	No	Detalle completo de una vacante. 404 si no existe.

El seed debe incluir al menos 90 vacantes con campos: id, title, company, city, salary (nullable), description, publishedAt, tags. Con limit=20 debe producir al menos 4 páginas. La paginación es por offset (page + limit). El endpoint debe soportar los siguientes valores de sort: date_desc (default), date_asc, salary_desc, salary_asc, relevance. salary_min y salary_max filtran por rango de salario; las vacantes con salary null se excluyen si alguno de los dos parámetros está presente.

Apply

Método	Endpoint	Auth	Descripción
POST	/jobs/:id/apply	Sí	Registra la aplicación. 409 si el usuario ya aplicó.
DELETE	/jobs/:id/apply	Sí	Cancela una aplicación existente. 404 si no existe.

Método	Endpoint	Auth	Descripción
GET	/applications	Sí	Lista de vacantes a las que aplicó el usuario autenticado.

Favoritos

Método	Endpoint	Auth	Descripción
POST	/jobs/:id/favorite	Sí	Agrega a favoritos. 409 si ya está.
DELETE	/jobs/:id/favorite	Sí	Elimina de favoritos. 404 si no existe.
GET	/favorites	Sí	Lista de favoritos del usuario autenticado.

2.3 Contrato de respuesta

Todos los endpoints responden con el mismo envelope:

```
// Éxito
{ "ok": true, "data": { ... } }

// Error
{ "ok": false, "error": { "code": "NOT_FOUND", "message": "Vacante no encontrada" } }
```

Códigos de error esperados: AUTH_REQUIRED (401), INVALID_CREDENTIALS (401), TOKEN_EXPIRED (401), NOT_FOUND (404), ALREADY_APPLIED (409), ALREADY_FAVORITED (409), VALIDATION_ERROR (422).

3. App móvil

App en Expo que consume la API del backend. Debe arrancar con `npx expo start` desde `app/` siguiendo las instrucciones del README.

Screen 1 · Job Search

Lista de vacantes consumida desde `GET /jobs`. Los controles de búsqueda, filtros y sorting se muestran inline en la pantalla, debajo de la barra de búsqueda, y actualizan el listado en tiempo real.

Barra de búsqueda

- Campo de texto libre que filtra por `q` (título y empresa)
- Debounce de 300ms antes de disparar la llamada al servidor

Filtros inline

- Selector de ciudad — filtra por `city`
- Rango de salario — dos inputs numéricos para `salary_min` y `salary_max`. Las vacantes sin salario se ocultan cuando alguno de los dos está activo

Sorting inline

- Selector con cinco opciones: Más recientes (date_desc · default), Más antiguos (date_asc), Mayor salario (salary_desc), Menor salario (salary_asc), Relevancia (relevance)
- Cambiar el sorting reinicia la paginación desde page 1

Lista

- Usar @shopify/flash-list
- Paginación: cargar más vacantes al llegar al final de la lista
- Cualquier cambio en filtros o sorting reinicia la lista desde el principio
- Estados de UI: skeleton loading, error de red con opción de reintentar, lista vacía

Screen 2 · Job Detail — Bottom Sheet con swipe

El detalle de una vacante se presenta como un `BottomSheet` que se abre sobre la lista. El usuario puede navegar entre vacantes haciendo swipe horizontal dentro del sheet. La pantalla también es alcanzable por deep link `occ://vacante/:id`.

Bottom Sheet

- Implementar con @gorhom/bottom-sheet usando BottomSheetModal
- Dos snap points: 60% de pantalla para el resumen y 100% para el detalle completo
- Cerrar el sheet no debe resetear el scroll de la lista subyacente
- Usar BottomSheetScrollView para el contenido interno

Swipe entre vacantes

- Swipe horizontal izquierda/derecha navega entre jobs de la lista
- Implementar con react-native-reanimated y react-native-gesture-handler
- La animación debe correr en el UI thread a 60 fps
- Al cerrar el sheet, el item activo debe ser visible en la lista

Paginación transparente durante el swipe

Cuando el usuario llega al último job de la página actual haciendo swipe, la app debe cargar silenciosamente la siguiente página en background y continuar la navegación sin interrupciones.

- El swipe no debe mostrar ningún estado de loading entre páginas
- La lista de búsqueda se actualiza en background de forma invisible mientras el sheet está abierto
- Si la carga de la página siguiente falla, el swipe se detiene suavemente con un indicador de fin de resultados
- Pre-cargar la página siguiente cuando el usuario esté a 3 jobs del final de la página actual

Acciones

- Botón Aplicar: llama a POST /jobs/:id/apply con el JWT en el header Authorization
- Botón Favorito (toggle): POST o DELETE /jobs/:id/favorite según el estado actual
- Si el token expiró (401), redirigir al login antes de ejecutar la acción
- Si ya aplicó (409), mostrar mensaje sin cerrar el sheet

Screen 3 · Auth

- Login que consume POST /auth/login y almacena el JWT en Zustand con persistencia en AsyncStorage
- Logout consume POST /auth/logout y limpia el store
- Al iniciar la app, validar el token con GET /auth/me antes de navegar
- Rutas protegidas con Expo Router: redirige a login si no hay sesión activa

Screen 4 · Mis actividades

Pantalla con los resultados de GET /applications y GET /favoritos del usuario autenticado. Puede ser una pantalla con tabs internos o dos pantallas separadas.

- Permitir cancelar una aplicación (DELETE /jobs/:id/apply) con feedback inmediato en la UI
- Permitir eliminar un favorito (DELETE /jobs/:id/favorite) con feedback inmediato en la UI

Screen 5 · Notificaciones push locales

Implementar con `@notifee/react-native` una notificación local que simule el mensaje 'Nueva vacante para ti'. Al tocarla, la app debe abrir el Bottom Sheet de Job Detail sobre la vacante correspondiente usando el deep link `occ://vacante/:id`.

- Configurar canal de notificación en Android con importancia HIGH
- Manejar el tap en los tres estados: foreground, background y quit (app cerrada)
- El payload de la notificación debe contener el id de la vacante
- Al abrir desde quit state, respetar el flujo de hidratación de sesión antes de navegar

Requisitos técnicos transversales

- TypeScript en modo strict. Sin any explícito. Tipos de respuesta de la API con Zod y `z.infer<>`.
- Capa de API centralizada en `app/src/services/api.ts`. No llamar a fetch o axios directamente desde los componentes.
- Interceptor que inyecte el JWT en el header Authorization de todas las llamadas autenticadas.
- URL base de la API desde variable de entorno, nunca hardcodeada.
- Al menos un test unitario por módulo core: store de sesión, hook de búsqueda y módulo de servicio de API.

4. Documentación arquitectónica

Todos los entregables van en la carpeta docs/ del monorepo como archivos Markdown. Es una parte tan importante como el código: nos interesa entender las decisiones que tomaste y por qué.

A1 · Arquitectura del monorepo

Diagrama o árbol de carpetas comentado para ambas partes del monorepo. Justificación del patrón elegido para la app y la estructura del backend. Criterio para decidir qué es compartido y qué queda encapsulado por feature.

A2 · Estrategia de estado y datos

Cómo manejas: sesión de usuario con JWT, caché de vacantes con invalidación selectiva, estado optimista en Apply y Favorito, y la estrategia de prefetch de jobs adyacentes para el swipe. Justificación de las herramientas elegidas y cuándo no las usarías.

A3 · Navegación y Deep Linking

Estructura de grupos de rutas con Expo Router, rutas protegidas y estrategia de deep linking desde notificaciones push. Incluir diagrama (ASCII o Mermaid). Cubrir el caso de abrir un deep link con la app cerrada y cómo resuelves la hidratación de sesión antes de navegar.

Incluir también: cómo coexisten el BottomSheetModal y el sistema de rutas, y cómo sincronizas el índice activo del swipe con la lista al cerrar el sheet.

A4 · Estrategia de calidad

Qué testeas en cada capa y con qué herramientas. Convenciones de commits y branching que adoptarías para el equipo. Qué debe cumplir un PR antes de ser aprobado.

A5 · Performance

Qué métricas monitorearías en producción, con qué herramientas y al menos dos optimizaciones concretas para la lista de vacantes. Incluir cómo gestionas el prefetch de jobs adyacentes sin impactar el render inicial.

5. Stack de referencia

El ejercicio está diseñado sobre las siguientes versiones. Puedes agregar librerías adicionales si lo consideras necesario, pero justifícalas en el README.

5.1 App

Paquete	Categoría	Notas
<code>expo-router ~6.0</code>	Navegación	Routing basado en sistema de archivos
<code>zustand ^4.5</code>	Estado	Estado global y persistencia de sesión
<code>@shopify/flash-list ^2.2</code>	Listas	Renderizado performante de listas largas
<code>react-native-reanimated ~4.1</code>	Animaciones	Animaciones en el UI thread
<code>@gorhom/bottom-sheet ^5</code>	UI	Bottom sheet del Job Detail
<code>react-native-gesture-handler ~2.28</code>	Gestos	Swipe entre vacantes
<code>zod ^3.24</code>	Validación	Tipado y validación de respuestas de API
<code>@notifee/react-native ^9</code>	Notificaciones	Notificaciones locales y manejo de eventos
<code>expo-constants ~18</code>	Config	Variables por ambiente

5.2 Backend

Paquete	Categoría	Notas
<code>express ^4</code>	Framework	Servidor HTTP y routing
<code>jsonwebtoken ^9</code>	Auth	Generación y verificación de JWT
<code>zod ^3</code>	Validación	Validación de request body y query params
<code>cors</code>	Config	Headers CORS para la app

Paquete	Categoría	Notas
<code>dotenv</code>	Config	Variables de entorno: PORT, JWT_SECRET

Cualquier duda sobre el alcance puedes escribirnos antes de comenzar.
